[1부] 머신러닝
5장 서포트 벡터 머신

핸즈온 머신러닝(3판)

ㅇㅇ대학교 ㅇㅇ학과
홍길동

시작하기 전에

지은이: 오렐리앙 제롱 Aurélien Géron

머신러닝 컨설턴트. 2013년에서 2016년까지 구글에서 유튜브 동영상 분류 팀을 이끌었다. 2002년에서 2012년까지 프랑스의 모바일 ISP 선두주자인 위퍼스트(Wifirst)를 설립하고 CTO로 일했다. 2001년에는 폴리콘셀(Polyconseil)을 설립하고 CTO로 일했다. 이 회사는 지금 전기차 공유 서비스인 오토립(Autolib')을 운영하고 있다.
그전에는 재무(J. P. 모건과 소시에테 제네랄(Société Générale)), 방위(캐나다 국방부), 의료(수혈) 등 다양한 분야에서 엔지니어로 일했고, C++, WiFi, 인터넷 구조에 관한 몇 권의 기술 서적을 썼으며 한 프랑스 공과대학에서 컴퓨터과학을 가르쳤다.

옮긴이: 박해선 haesun.park@tensorflow.blog

기계공학을 전공했지만 졸업 후엔 줄곧 코드를 읽고 쓰는 일을 했다. 블로그(*tensorflow.blog*)에 글을 쓰고 머신러닝과 딥러닝에 관한 책을 집필, 번역하면서 소프트웨어와 과학의 경계를 흥미롭게 탐험하고 있다.
『챗GPT로 대화하는 기술』(한빛미디어, 2023), 『혼자 공부하는 머신러닝+딥러닝』(한빛미디어, 2020), 『혼자 공부하는 데이터 분석 with 파이썬』(한빛미디어, 2023), 『Do it! 딥러닝 입문』(이지스퍼블리싱, 2019) 등 집필했다.

[코드 예제] *https://bit.ly/homl3-git* 에서 주피터 노트북으로 제공

3판의 주요 변경 내용

1. 최신 라이브러리 버전으로 전체 코드 업데이트
2. 특성 이름 추적, 히스토그램 기반 그레이디언트 부스팅, 레이블 전파 등 사이킷런에 새롭게 추가된 다양한 기능
3. 하이퍼파라미터 튜닝을 위한 케라스 튜너(Keras Tuner) 라이브러리, 자연어 처리를 위한 허깅 페이스(Hugging Face)의 트랜스포머스(Transformers) 라이브러리 및 케라스의 새로운 전처리 및 데이터 증식 층
4. 여러 비전 모델(ResNeXt, DenseNet, MobileNet, CSPNet, EfficientNet)과 올바른 모델을 선택하기 위한 가이드라인
5. <15장 RNN과 CNN을 사용한 시퀀스 처리>: 합성된 시계열 대신 시카고 버스 및 철도 탑승객 데이터를 분석하며 ARMA 모델과 그 변형
6. <16장 RNN과 어텐션을 사용한 자연어 처리>: 인코더-디코더 RNN, 트랜스포머 모델을 사용한 영어-스페인어 번역 모델 구축, 스위치 트랜스포머(Switch Transformer), DistilBERT, T5, PaLM(사고 사슬 프롬프트 포함)과 같은 언어 모델 비전 트랜스포머(ViT) 소개, DeiT, 퍼시비어, DINO와 같은 트랜스포머 기반 비전 모델을 비롯한 CLIP, DALL·E, 플라밍고, GATO 등 몇 가지 대형 멀티모달 모델 개요
7. <17장 오토인코더, GAN 그리고 확산 모델>: 확산 모델 소개, DDPM 구현
8. <19장 대규모 텐서플로 모델 훈련과 배포>: 구글 클라우드 AI 플랫폼에서 구글 버텍스 AI로 마이그레이션, 대규모 하이퍼파라미터 검색을 위한 분산 케라스 튜너, TensorFlow.js 코드
9. PipeDream과 Pathways를 비롯한 추가적인 분산 훈련 기법

[참조] https://homl.info/changes3

이 책의 학습 목표

1부 머신러닝

1장: 한눈에 보는 머신러닝	데이터 과학자가 꼭 알아야 할 기초 개념과 용어
2장: 머신러닝 프로젝트 처음부터 끝까지	주택 가격을 예측하는 회귀 작업을 살펴보면서 선형 회귀, 결정 트리, 랜덤 포레스트 등 여러 알고리즘 학습
3장: 분류	분류 시스템 학습
4장: 모델 훈련	신경망 구축에 필요한 모델 훈련 알고리즘 학습
5장: 서포트 벡터 머신	SVM의 핵심 개념, 사용 방법, 작동 원리 학습
6장: 결정 트리	결정 트리의 훈련, 시각화, 예측 방법과 사이킷런의 CART 훈련 알고리즘 등
7장: 앙상블 학습과 랜덤 포레스트	투표 기반 분류기, 배깅과 페이스팅 앙상블, 랜덤 포레스트, 부스팅, 스태킹 앙상블 등
8장: 차원 축소	고차원 공간과 차원 축소 기법
9장: 비지도 학습	비지도 학습과 알고리즘

이 책의 학습 목표

2부 신경망과 딥러닝

10장: 케라스를 사용한 인공 신경망 소개	인공 신경망과 케라스를 이용한 구현 방법
11장: 심층 신경망 훈련	심층 신경망의 문제와 해결 방법
12장: 텐서플로를 사용한 사용자 정의 모델과 훈련	텐서플로와 저수준 파이썬 API를 이용한 사용자 정의 모델과 훈련 알고리즘
13장: 텐서플로를 사용한 데이터 적재와 전처리	tf.data API와 TFRecord 포맷, 케라스 전처리 층 및 tf.data API 사용 방법
14장: 합성곱 신경망을 사용한 컴퓨터 비전	CNN 구성 요소, 케라스를 사용한 구현 방법
15장: RNN과 CNN을 사용한 시퀀스 처리	RNN 개념과 WaveNet 구현
16장: RNN과 어텐션을 사용한 자연어 처리	문장 수준의 RNN과 어텐션 메커니즘
17장: 오토인코더, GAN 그리고 확산 모델	오토인코더 차원 축소, 특성 추출, 비지도 사전 훈련 방법과 생성 모델
18장: 강화 학습	강화 학습 개념, 정책 그레이디언트, 심층 Q-네트워크
19장: 대규모 텐서플로 모델 훈련과 배포	TF 서빙과 구글 버텍스 AI 플랫폼에 모델을 배포하는 방법

이 책의 학습 목표

3부 부록

부록 A: 연습문제 정답

부록 B: 머신러닝 프로젝트 체크리스트

부록 C: 자동 미분

부록 D: 특수한 데이터 구조

부록 E: 텐서플로 그래프

Contents

- 5장: 서포트 벡터 머신

5.1 선형 SVM 분류
5.2 비선형 SVM 분류
5.3 SVM 회귀
5.4 SVM 이론
5.5 쌍대 문제

5장 서포트 벡터 머신

SVM의 핵심 개념, 사용 방법, 작동 원리 학습

5.1 선형 SVM 분류(1)

- 서포트 벡터 머신(SVM)
  - 매우 강력하고 선형이나 비선형 분류, 회귀, 이상치 탐색에도 사용할 수 있는 다목적 머신러닝 모델
    - 라지 마진 분류(large margin classification)
    - 서포트 벡터(support vector)

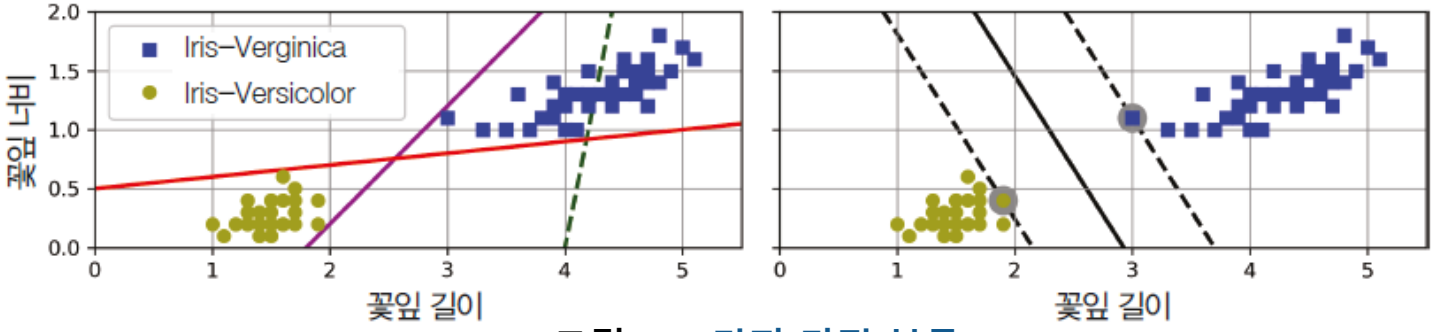

그림 5-1 라지 마진 분류

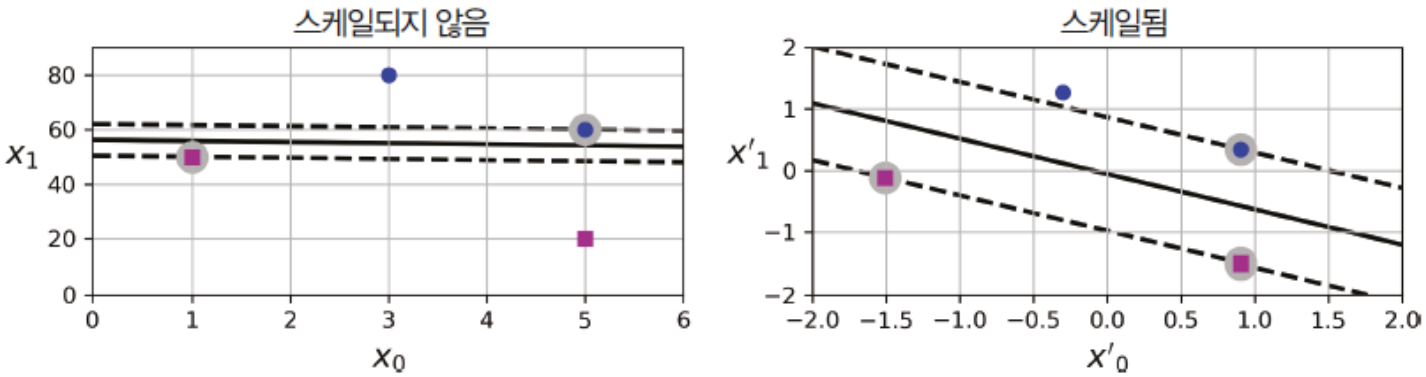

그림 5-2 특성 스케일에 따른 민감성

5.1 선형 SVM 분류(2)

5.1.1 소프트 마진 분류

- 하드 마진 분류(hard margin classification)의 두 가지 문제점
  - 데이터가 선형적으로 구분될 수 있어야 제대로 작동
  - 이상치에 민감
- 소프트 마진 분류(soft margin classification)
  - 도로의 폭을 가능한 한 넓게 유지하는 것과 마진 오류)margin violation) 사이에 적절한 균형을 유지

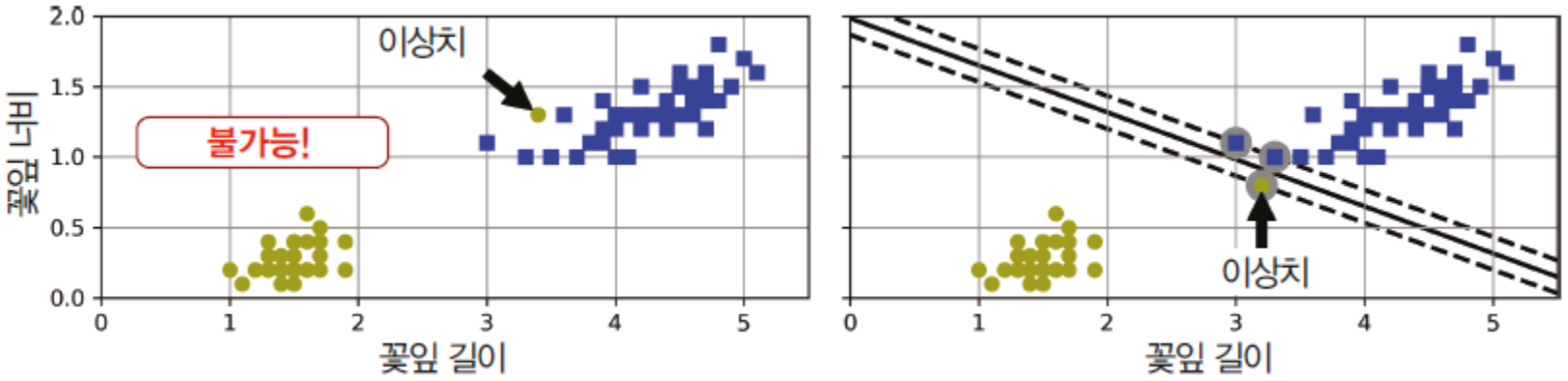

그림 5-3 이상치에 민감한 하드 마진

5.1 선형 SVM 분류(3)

- SVM 모델을 만들 때 규제 하이퍼파라미터 C를 포함하여 여러 하이퍼파라미터를 지정
  - C를 줄이면 도로가 더 커지지만 더 많은 마진 오류가 발생
    - 도로를 지지하는 샘플이 더 많아지므로 과대적합의 위험이 줄어듬
  - 너무 많이 줄이면 이 경우처럼 모델이 과소적합. 즉, C=100인 모델이 C=1인 모델보다 더 잘 일반화
  - SVM 모델이 과대적합이라면 C를 감소시켜 모델을 규제

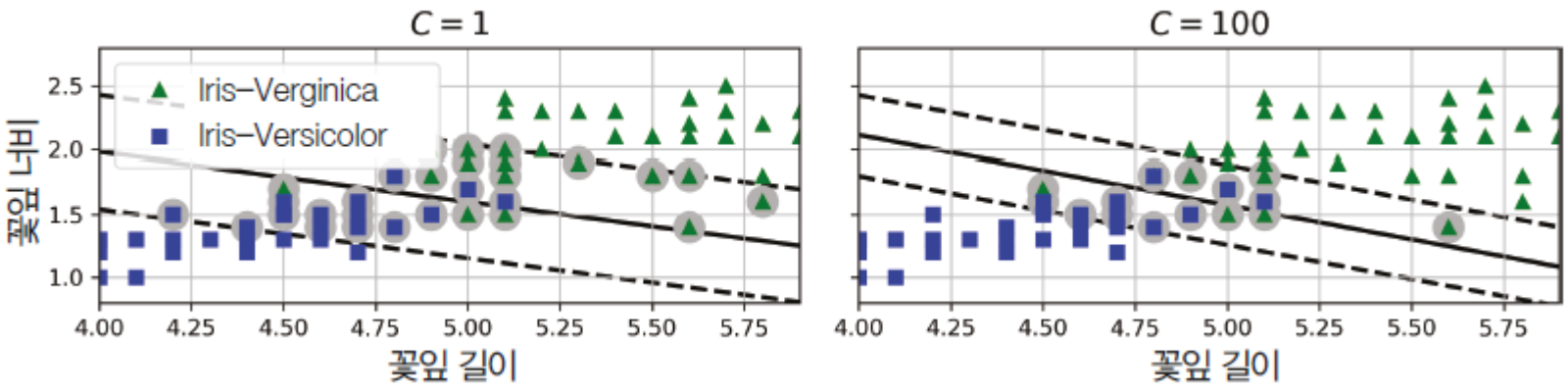

그림 5-4 넓은 마진(왼쪽) vs. 적은 마진 오류(오른쪽)

5.1 선형 SVM 분류(4)

- 붓꽃 데이터셋을 적재하고, Iris-Virginia 품종을 감지하기 위해 선형 SVM 모델을 훈련
  - 이 파이프라인은 먼저 특성의 스케일을 변경한 다음 C =1인 LinearSVC를 사용
  - [그림 5-4]의 오른쪽 그래프

```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2) # Iris-Virginica

svm_clf = make_pipeline(StandardScaler(),
                        LinearSVC(C=1, random_state=42))
svm_clf.fit(X, y)
```

5.1 선형 SVM 분류(5)

- 모델을 사용해 예측

```
>>> X_new = [[5.5, 1.7], [5.0, 1.5]]
>>> svm_clf.predict(X_new)
array([ True, False])
```

- SVM이 이러한 예측을 하는 데 사용한 점수
  - SVM 모델은 각 샘플과 결정 경계 사이의 거리(양수 또는 음수)를 측정

```
>>> svm_clf.decision_function(X_new)
array([ 0.66163411, -0.22036063])
```

5.2 비선형 SVM 분류(1)

- 비선형 데이터셋에 다항 특성과 같은 특성을 더 추가하여 선형적으로 구분되는 데이터셋 만들기

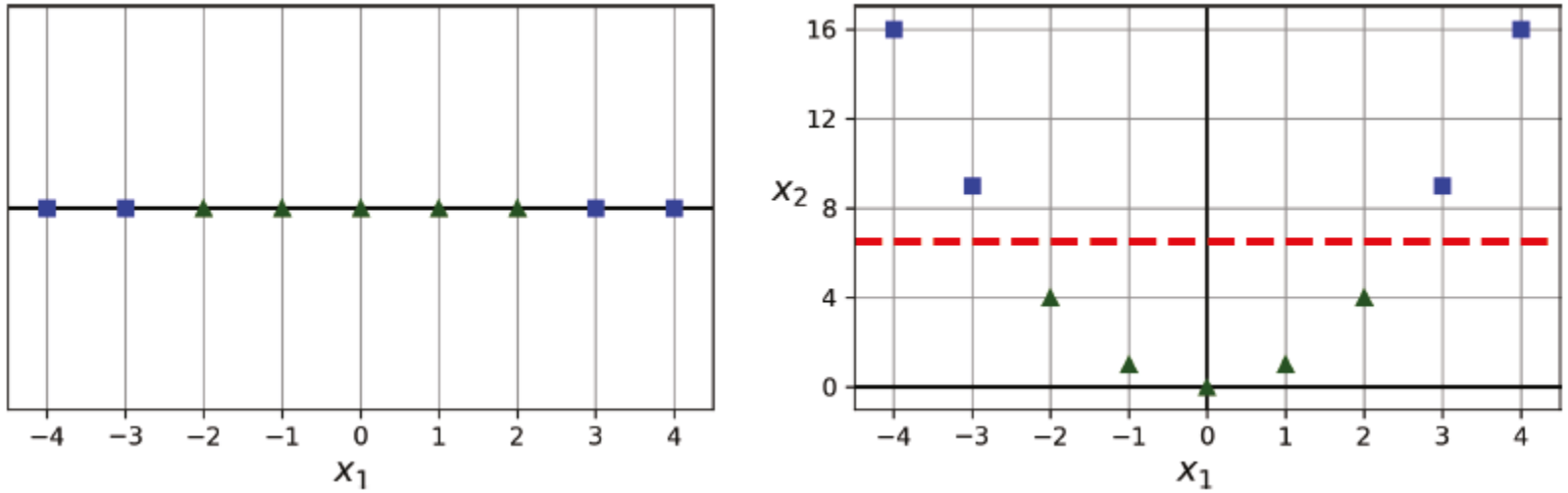

그림 5-5 특성을 추가하여 선형적으로 구분되는 데이터셋 만들기

5.2 비선형 SVM 분류(2)

- PolynomialFeatures 변환기와 StandardScaler, LinearSVC를 연결하여 파이프라인을 만들고 moons 데이터셋에 적용
  - make_moons() 함수를 사용

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures

X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

polynomial_svm_clf = make_pipeline(
    PolynomialFeatures(degree=3),
    StandardScaler(),
    LinearSVC(C=10, max_iter=10_000, random_state=42)
)
polynomial_svm_clf.fit(X, y)
```

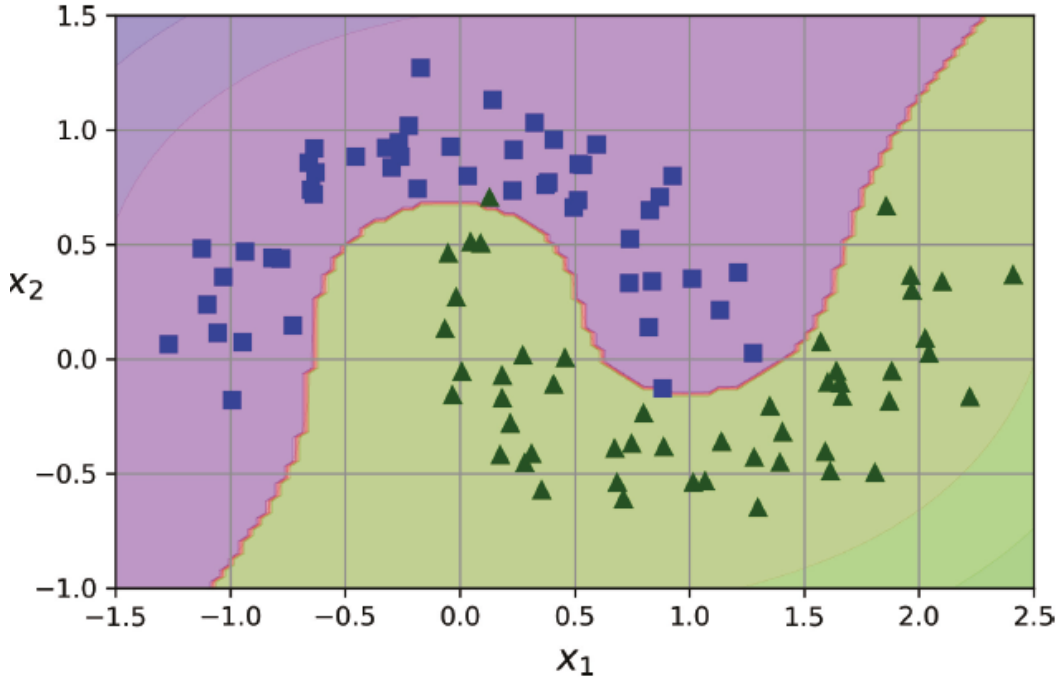

그림 5-6 다항 특성을 사용한 선형 SVM 분류기

5.2 비선형 SVM 분류(3)

5.2.1 다항식 커널

- 커널 트릭(kernel trick)
  - 실제로는 특성을 추가하지 않으면서 매우 높은 차수의 다항 특성을 많이 추가한 것과 같은 결과 도출

```
from sklearn.svm import SVC

poly_kernel_svm_clf = make_pipeline(StandardScaler(),
                                    SVC(kernel="poly", degree=3, coef0=1, C=5))
poly_kernel_svm_clf.fit(X, y)
```

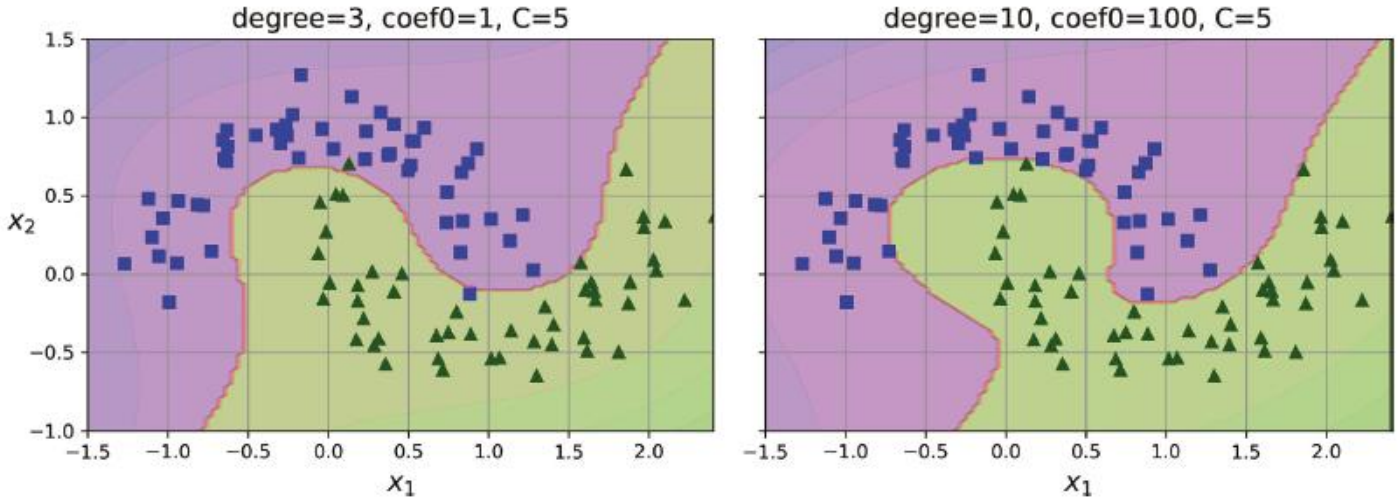

그림 5-7 다항식 커널을 사용한 SVM 분류기

5.2 비선형 SVM 분류(4)

5.2.2 유사도 특성

- 유사도 함수(similarity function)로 계산한 특성을 추가 - 랜드마크와 얼마나 닮았는지 측정
  - 앞에서 본 1차원 데이터셋에 두 개의 랜드마크 x1 = -2와 x1 = 1을 추가([그림 5-8]의 왼쪽 그래프)
  - γ = 0.3인 가우스 방사 기저 함수(radial basis function, RBF)를 유사도 함수로 정의

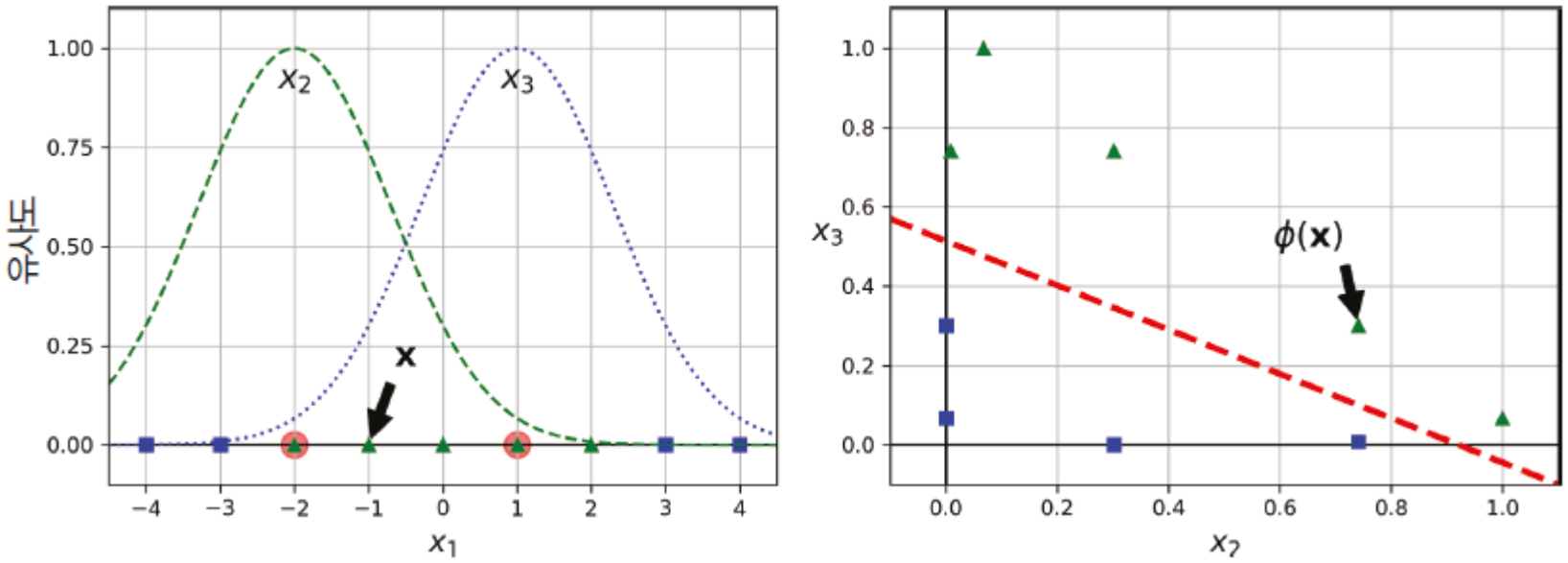

그림 5-8 가우스 RBF를 사용한 유사도 특성

5.2 비선형 SVM 분류(5)

5.2.3 가우스 RBF 커널

- 가우스R BF 커널을 사용한 SVC 모델

```
rbf_kernel_svm_clf = make_pipeline(StandardScaler(),
                                   SVC(kernel="rbf", gamma=5, C=0.001))
rbf_kernel_svm_clf.fit(X, y)
```

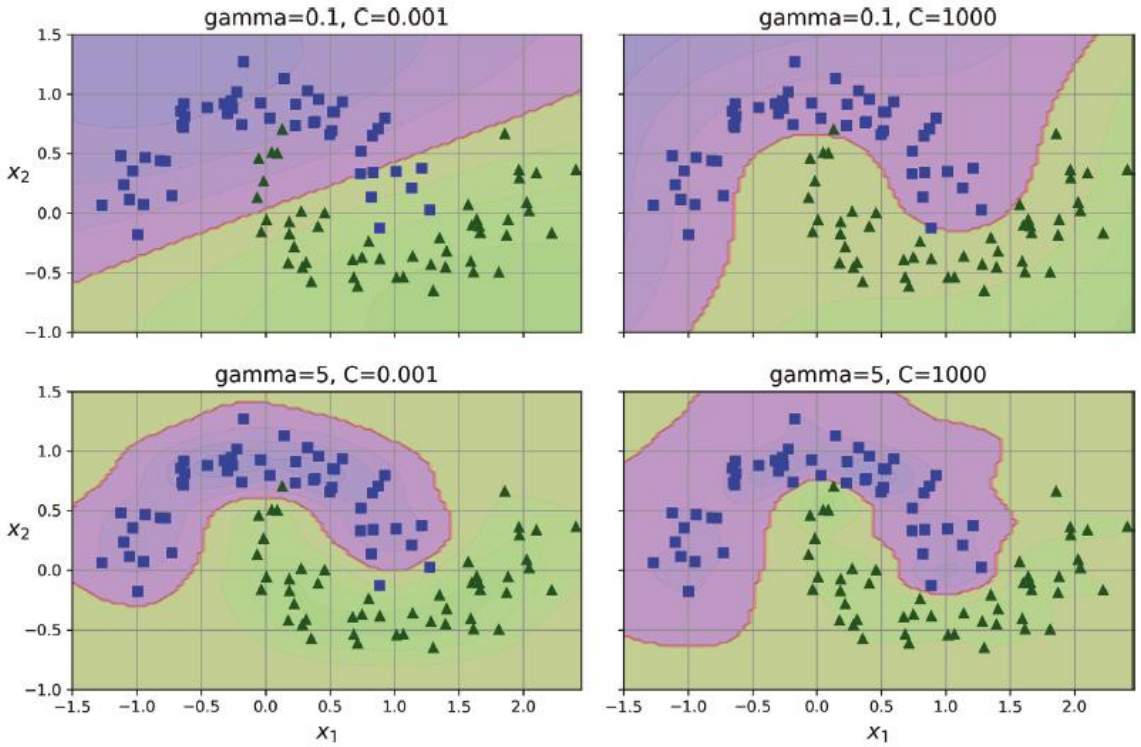

그림 5-9 RBF 커널을 사용한 SVM 분류기

5.2 비선형 SVM 분류(6)

- 문자열 커널(string kernel)
  - 텍스트 문서나 DNA 서열을 분류할 때 사용
  - 문자열 서브시퀀스 커널(string subsequence kernel)이나 레벤슈타인 거리(Levenshtein distance) 기반의 커널

- 여러 가지 커널 중 어떤 것을 사용해야 할까?
  - 경험적으로 봤을 때 언제나 선형 커널을 가장 먼저 시도
    - LinearSVC가 SVC(kernel="linear")보다 훨씬 빠르다는 것을 기억
    - 특히 훈련 세트가 아주 큰 경우
  - 훈련 세트가 너무 크지 않다면 가우스 RBF 커널도 시도
  - 시간과 컴퓨팅 성능이 충분하다면 교차 검증과 하이퍼파라미터 탐색을 사용해 다른 커널도 시도

5.2 비선형 SVM 분류(7)

5.2.4 계산 복잡도

표 5-1 SVM 분류를 위한 사이킷런 파이썬 클래스 비교

파이썬 클래스	시간 복잡도	외부 메모리 학습 지원	스케일 조정의 필요성	커널 트릭
LinearSVC	$O(m \times n)$	아니오	예	아니오
SVC	$O(m^2 \times n) \sim O(m^3 \times n)$	아니오	예	예
SGDClassifier	$O(m \times n)$	예	예	아니오

5.3 SVM 회귀(1)

- SVM을 분류가 아니라 회귀에 적용하는 방법
  - 일정한 마진 오류 안에서 두 클래스 간의 도로 폭이 가능한 한 최대가 되도록 하는 대신, SVM 회귀는 제한된 마진 오류(즉, 도로 밖의 샘플) 안에서 도로 안에 가능한 한 많은 샘플이 들어가도록 학습
  - 도로의 폭은 하이퍼파라미터 ε으로 조절

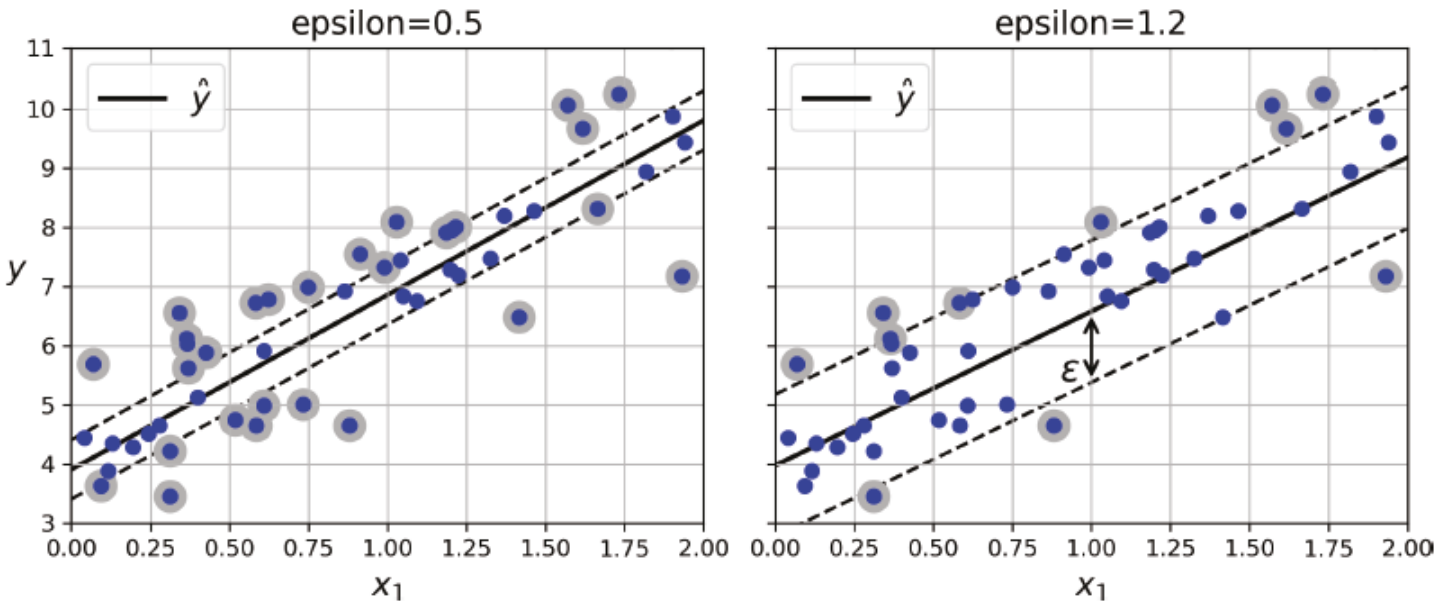

그림 5-10 SVM 회귀

5.3 SVM 회귀(2)

- LinearSVR을 사용해 선형 SVM 회귀를 적용

```
from sklearn.svm import LinearSVR

X, y = [...] # 선형 데이터셋
svm_reg = make_pipeline(StandardScaler(),
                        LinearSVR(epsilon=0.5, random_state=42))
svm_reg.fit(X, y)
```

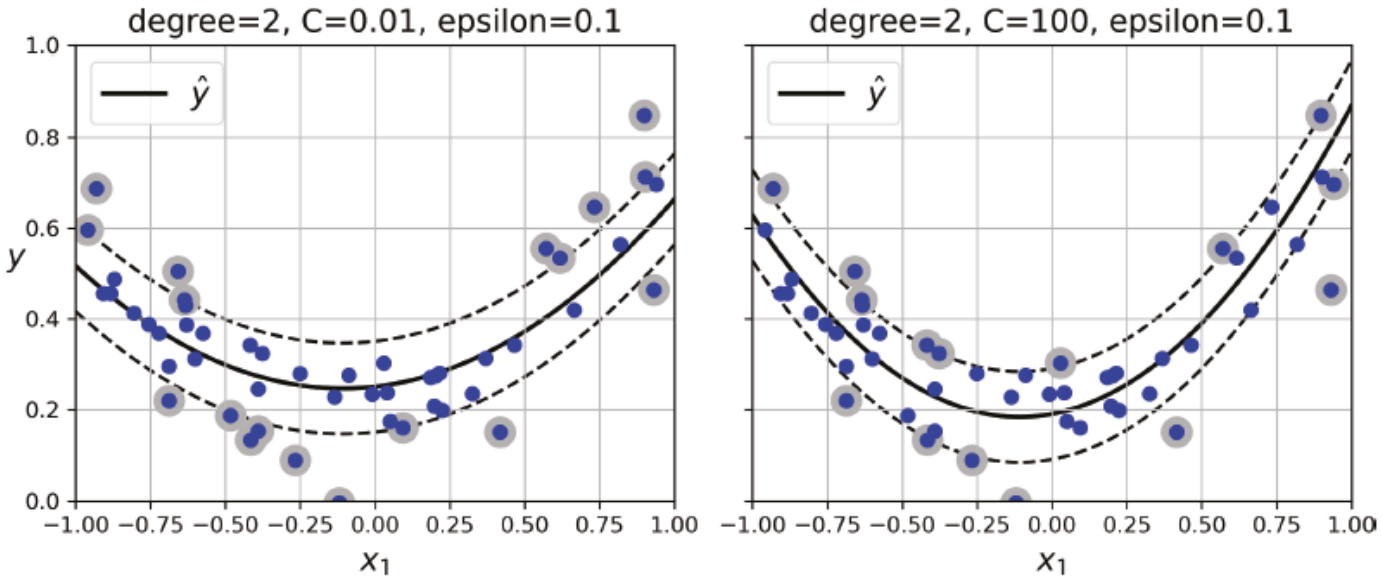

그림 5-11 2차 다항 커널을 사용한 SVM 회귀

5.3 SVM 회귀(3)

- 사이킷런의 SVR을 사용해 [그림 5-11]의 왼쪽 그래프에 해당하는 모델을 생성

```
from sklearn.svm import SVR

X, y = [...] # 2차 방정식 데이터셋
svm_poly_reg = make_pipeline(StandardScaler(),
                             SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1))
svm_poly_reg.fit(X, y)
```

- SVR은 SVC의 회귀 버전이고 LinearSVR은 LinearSVC의 회귀 버전
- LinearSVR은 (LinearSVC처럼) 필요한 시간이 훈련 세트의 크기에 비례해서 선형적으로 늘어남
- 하지만 SVR은 (SVC처럼) 훈련 세트가 커지면 훨씬 느려짐

5.4 SVM 이론(1)

- 선형 SVM 분류기 모델은 결정 함수 $\theta^T x = \theta_0 x_0 + \ldots + \theta_n x_n$를 계산해서 새로운 샘플 x의 클래스를 예측
  - (x_0는 편향 특성이며 항상 1)
  - 결괏값이 0보다 크면 예측된 클래스 $\hat{y}$은 양성 클래스(1)가 됨
  - 그렇지 않으면 음성 클래스(0)가 됨
- 선형 SVM 분류기로 훈련
  - 마진 오류 횟수를 제한하면서 도로(또는 마진)를 가능한 한 넓게 만드는 가중치 벡터 w와 편향 b를 찾음

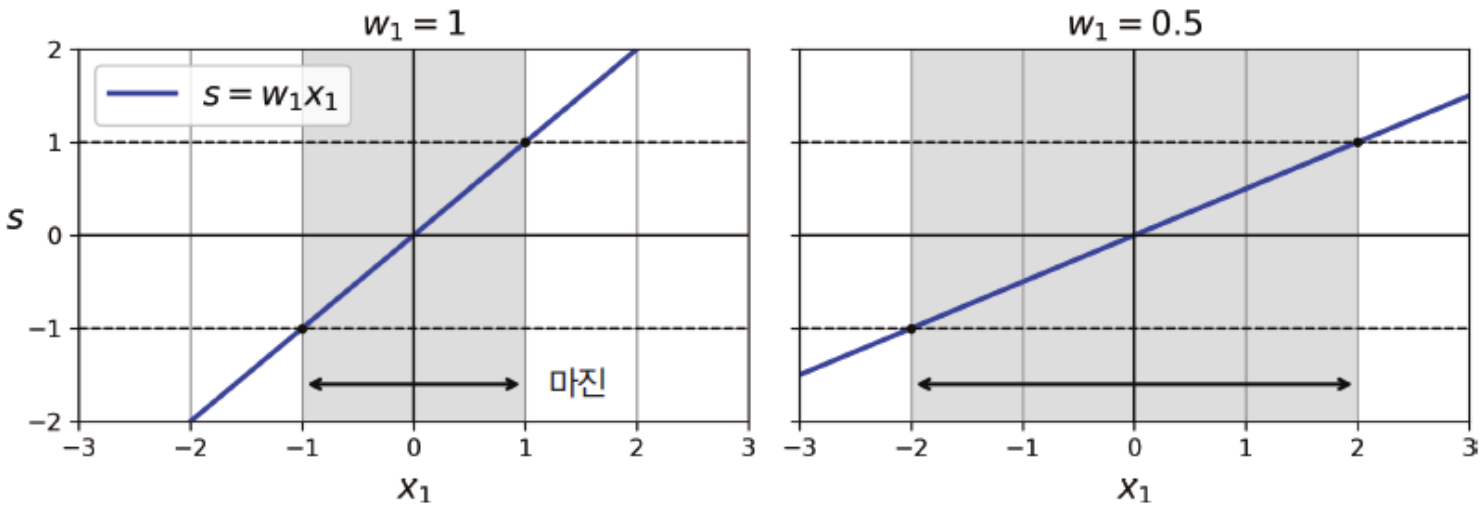

그림 5-12 가중치 벡터가 작을수록 마진은 커짐

5.4 SVM 이론(2)

- 하드 마진 선형 SVM 분류기의 목적 함수를 제약이 있는 최적화(constrained optimization) 문제로 표현

식 5-1 하드 마진 선형 SVM 분류기의 목적 함수

$$\underset{\mathbf{w},b}{\text{minimize}}\ \frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w}$$

[조건] $i=1,2,\cdots,m$일 때 $t^{(i)}\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}^{(i)}+b\right)\geq 1$

- 소프트 마진 분류기의 목적 함수를 구성하려면 각 샘플에 대해 슬랙 변수(slack variable) $\zeta^{(i)} \geq 0$을 도입

식 5-2 소프트 마진 선형 SVM 분류기의 목적 함수

$$\underset{\mathbf{w},b,\zeta}{\text{minimize}}\ \frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w}+C\sum_{i=1}^{m}\zeta^{(i)}$$

[조건] $i=1,2,\cdots,m$일 때 $t^{(i)}\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}^{(i)}+b\right)\geq 1-\zeta^{(i)}$이고 $\zeta^{(i)}\geq 0$

5.4 SVM 이론(3)

- 경사 하강법을 사용하여 힌지 손실(hinge loss) 또는 제곱 힌지 손실(squared hinge loss)을 최소화

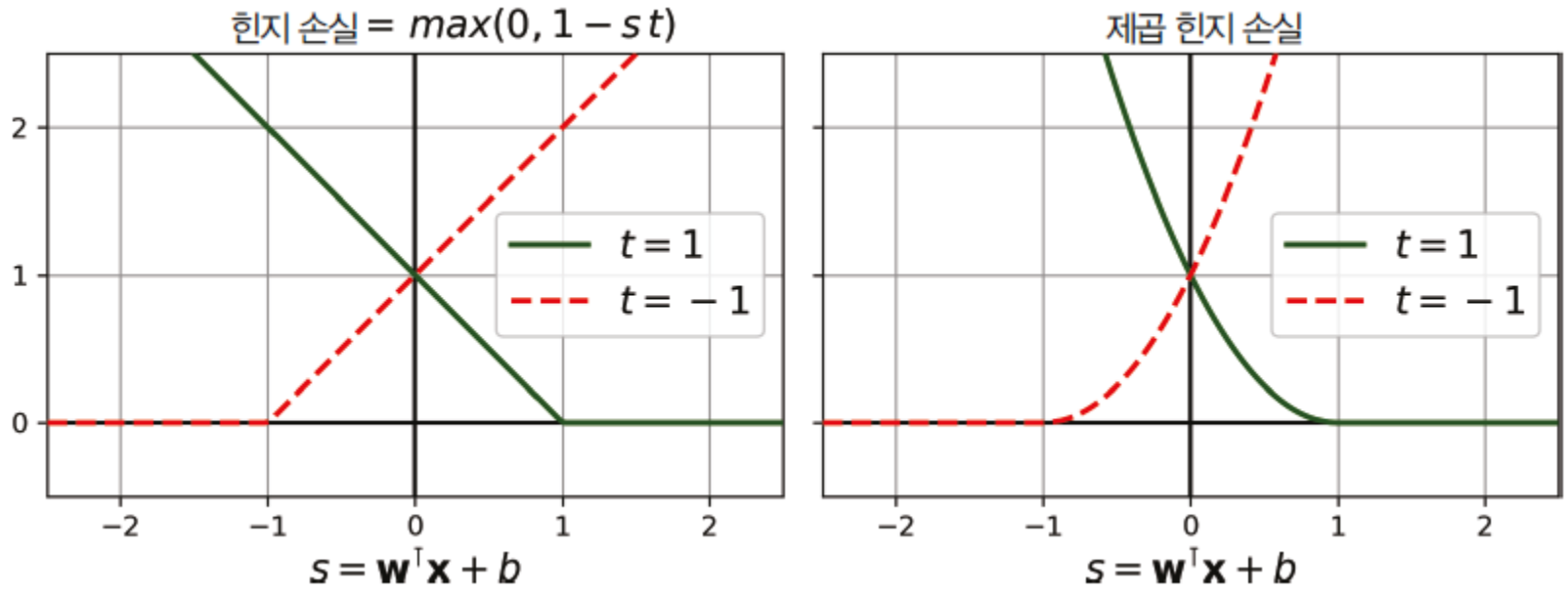

그림 5-13 힌지 손실(왼쪽)과 제곱 힌지 손실(오른쪽)

5.5 쌍대 문제(1)

- 일반적으로 쌍대 문제의 해는 원 문제 해의 하한값이지만 어떤 조건하에서는 원 문제와 똑같은 해를 제공
- SVM 문제는 이 조건을 만족시킴

식 5-3 선형 SVM 목적 함수의 쌍대 형식

$$\underset{\alpha}{\text{minimize}}\ \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\mathbf{x}^{(i)\mathrm{T}}\mathbf{x}^{(j)}-\sum_{i=1}^{m}\alpha^{(i)}$$

[조건] $i=1, 2, \cdots, m$에 대해서 $\alpha^{(i)}\geq 0$ 이고 $\sum_{i=1}^{m}\alpha^{(i)}t^{(i)}=0$

식 5-4 쌍대 문제에서 구한 해로 원 문제의 해 계산하기

$$\hat{\mathbf{w}}=\sum_{i=1}^{m}\hat{\alpha}^{(i)}t^{(i)}\mathbf{x}^{(i)}$$

$$\hat{b}=\frac{1}{n_s}\sum_{\substack{i=1\\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)}-\hat{\mathbf{w}}^{\mathrm{T}}\mathbf{x}^{(i)}\right)$$

5.5 쌍대 문제(2)

5.5.1 커널 SVM

- 모든 훈련 샘플에 변환 φ를 적용하면 쌍대 문제(식 5-3)에 점곱 $\varphi(x^{(i)})^T\varphi(x^{(j)})$가 포함
  - 하지만 ϕ가 [식 5-5]에 정의된 2차 다항식 변환이라면 변환된 벡터의 점곱을 간단하게 $(x^{(i)T}x^{(j)})^2$ 으로 바꿀 수 있음
  - 그래서 실제로 훈련 샘플을 변환할 필요가 전혀 없음. 즉, [식 5-3]에 있는 점곱을 제곱으로 바꾸기만 하면 됨

식 5-5 2차 다항식 매핑

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{pmatrix}$$

식 5-6 2차 다항식 매핑을 위한 커널 트릭

$$\phi(\mathbf{a})^T\phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}a_1a_2 \\ a_2^2 \end{pmatrix}^T \begin{pmatrix} b_1^2 \\ \sqrt{2}b_1b_2 \\ b_2^2 \end{pmatrix} = a_1^2b_1^2 + 2a_1b_1a_2b_2 + a_2^2b_2^2$$

$$= (a_1b_1 + a_2b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}\right)^2 = (\mathbf{a}^T\mathbf{b})^2$$

5.5 쌍대 문제(3)

- 커널 트릭에서 $\hat{w}$ 을 모른 채로 예측 만들기

식 5-8 커널 SVM으로 예측하기

$$h_{\hat{\mathbf{w}}\hat{b}}\left(\phi\left(\mathbf{x}^{(n)}\right)\right) = \hat{\mathbf{w}}^{\mathrm{T}}\phi\left(\mathbf{x}^{(n)}\right) + \hat{b} = \left(\sum_{i=1}^{m}\hat{\alpha}^{(i)}t^{(i)}\phi\left(\mathbf{x}^{(i)}\right)\right)^{\mathrm{T}}\phi\left(\mathbf{x}^{(n)}\right) + \hat{b}$$

$$= \sum_{i=1}^{m}\hat{\alpha}^{(i)}t^{(i)}\left(\phi\left(\mathbf{x}^{(i)}\right)^{\mathrm{T}}\phi\left(\mathbf{x}^{(n)}\right)\right) + \hat{b}$$

$$= \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)}>0}}^{m}\hat{\alpha}^{(i)}t^{(i)}K\left(\mathbf{x}^{(i)}, \mathbf{x}^{(n)}\right) + \hat{b}$$

식 5-9 커널 트릭을 사용한 편향 계산

$$\hat{b} = \frac{1}{n_s}\sum_{\substack{i=1 \\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \hat{\mathbf{w}}^{\mathrm{T}}\phi\left(\mathbf{x}^{(i)}\right)\right) = \frac{1}{n_s}\sum_{\substack{i=1 \\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \left(\sum_{j=1}^{m}\hat{\alpha}^{(j)}t^{(j)}\phi\left(\mathbf{x}^{(j)}\right)\right)^{\mathrm{T}}\phi\left(\mathbf{x}^{(i)}\right)\right)$$

$$= \frac{1}{n_s}\sum_{\substack{i=1 \\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \sum_{\substack{j=1 \\ \hat{\alpha}^{(j)}>0}}^{m}\hat{\alpha}^{(j)}t^{(j)}K\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right)\right)$$

연습문제

1. 서포트 벡터 머신의 근본 아이디어는 무엇인가?
2. 서포트 벡터가 무엇인가?
3. SVM을 사용할 때 입력값의 스케일이 왜 중요한가?
4. SVM 분류기가 샘플을 분류할 때 신뢰도 점수와 확률을 출력할 수 있나?
5. LinearSVC, SVC, SGDClassifier 중에서 어떤 것을 선택해야 하나?
6. RBF 커널을 사용해 SVM 분류기를 훈련시켰더니 훈련 세트에 과소적합된 것 같다. γ(gamma)를 증가시켜야 할까, 감소시켜야 할까? C의 경우는 어떤가?
7. 모델이 ε에 민감하다는 것은 무슨 의미인가?
8. 커널 트릭을 사용하는 이유는?
9. 선형적으로 분리되는 데이터셋에 LinearSVC를 훈련시키고, 그런 다음 같은 데이터셋에 SVC와 SGDClassifier를 적용해보기. 거의 비슷한 모델이 만들어지는지 확인.
10. sklearn.datasets.load_wine()을 사용해 로드할 수 있는 와인 데이터셋에서 SVM 분류기를 훈련하기
11. 캘리포니아 주택 가격 데이터셋에 SVM 회귀를 훈련시켜보기